Introduction to R

Instructors: Maryam Movahedifar, Susanne de Vogel

Table of Contents

What is R?

R is an interpreted programming language for statistical computing and analysis.

Some of the features of this language include:

- It is free.
- It has simple and intuitive commands that make it easy to use.
- It provides powerful and diverse packages for plotting, statistical analysis, and statistical computations.

- It has a strong user community.
- And more...

In recent years, due to its statistical and graphical capabilities and its ease of use, R has become one of the most popular languages among statisticians, data analysts, researchers, and others.

Introduction to R

Installing R

First, download R from here.

After installing R, run it.

In the console, execute the following command to download and install the required packages.

In [1]:
```
install.packages(c('repr', 'IRdisplay', 'evaluate', 'crayon', 'pbdZMQ', 'devtools', 'uuid', 'digest'))
```

```
Updating HTML index of packages in '.Library'

Making 'packages.html' ...
 done
```

Installing RStudio:

RStudio is a graphical environment for working with R. It is recommended to install it for solving exercises and learning R.

Download it from here.

R Packages

Packages in R consist of a collection of functions and datasets.

Installing Packages:

The most common way to install a package in R is as follows:

In [ ]:
```
install.packages("Name Of the package")
```

Other methods for installing packages also exist, but they are not the focus for us at this time.

Loading Packages:

To use any package, you must load it into R.

In [ ]:
```
library()
```

Other Useful Commands Related to Packages:

In [ ]:
```
## Removing a package
remove.packages("")

## Installed packages that need to be updated
old.packages()

## Updating old packages
update.packages()

## Using a function from a package without loading the package
### PackageName::function()
ggplot2::geom_point
```

R Basics

We will start working with R in the most effective way for learning:

In [2]:
```
print("Hello World!")
```

```
[1] "Hello World!"
```

Printing:

Suppose we want to display the value of an expression or variable. To do this, simply execute the expression or variable, and the result will be printed.

In [3]:
```
1 + 1
pi
sqrt(8)
```

```
2
3.14159265358979
2.82842712474619
```

We can also use the `print()` function to display output.

In [4]:
```
print(pi)
```

```
[1] 3.141593
```

The distinguishing feature of the `print()` function is that it prints any object in R in the correct format.
For example, below we print a matrix (we will discuss data structures later).

In [5]:
```
print(matrix(c(1,2,3,4),2,2))
```

```
     [,1] [,2]
[1,]    1    3
[2,]    2    4
```

Note: The `print()` function prints only one object at a time.

In [6]:
```
print("Pi number is equal to " , pi)
```

```
[1] "Pi number is equal to "
```

For this purpose, we use the `cat()` function.

In [7]:
```
cat("Pi number is equal to", pi , "\n")
```

```
Pi number is equal to 3.141593
```

The `cat()` function can also print simple vectors.

In [8]:
```
cat("12 months of the year are" , month.name , "\n")
```

```
12 months of the year are January February March April May June July August September October November December
```

(`month.name` and `pi` are examples of constants provided by R.)

Defining a Variable:

In R, you do not need to specify the type of a variable. You can simply use the `<-` operator.

In [9]:
```
s <- "this is a string"
x <- 4
s
print(x)
```

```
'this is a string'
[1] 4
```

Note 1: When naming objects, you can use a period `"."` in the name.

For example, `month.name` is the name of a vector in R.

Note 2: You can also use `=` for assignment.

However, in R, the `=` operator also has another purpose: specifying a parameter in a function.

For this reason, it is generally better to use `<-` for assignments.

Learn more about the differences here.

By defining a variable in this way, it will be stored in the workspace.

- To see all variables and functions in your workspace, use `ls()`.
- To get detailed information about objects, use `ls.str()`.

In [10]:
```
ls()
ls.str()
a <- 4
ls()
ls.str()
```

```
's' · 'x'
 s :  chr "this is a string"
 x :  num 4
'a' · 's' · 'x'
 a :  num 4
 s :  chr "this is a string"
 x :  num 4
```

You can use the `rm()` function to remove variables from your workspace.

In [11]:
```
print("Removing x")
rm("x")
ls.str()

##Clearing the workspace
rm(list = ls())

print("The workspace now :")
ls()
```

```
[1] "Removing x"
a :  num 4
s :  chr "this is a string"
[1] "The workspace now :"
```

Getting Help in R:

One of the best ways to learn and get help is by studying the documentation for packages and functions, which is available directly in R.

You can view this documentation using the following methods:

In [ ]:
```
## Documentation of installed functions
?mean

## Documentation of functions
??highchart

## Documentation of packages or functions
help("")

#### Getting help on packages
vignette("")
```

Creating a Vector:

Vectors are one of the most fundamental elements in R.

Vectors can contain numbers, strings, or logical values, but they cannot be a mix of different types.

You can create a vector using the `c()` function.

In [12]:
```
c(1,3,3*pi,4+5)
c(T,F,T)
```

1 · 3 · 9.42477796076938 · 9

TRUE · FALSE · TRUE

If the values provided to `c()` are themselves vectors, it will flatten them and combine them with the other values.

In [13]:
```
v1 <- c(1,4,5)
v2 <- c(8,5,6)
c(2,v1,v2,0)
```

2 · 1 · 4 · 5 · 8 · 5 · 6 · 0

Comparing Two Vectors:

When comparing two vectors, R performs an element-wise comparison and returns a vector of logical values (TRUE or FALSE).

In [14]:
```
v <- c(2,4,6)
u <- c(3,4,5)

u != v   # 2 != 3  4 == 4  6 != 5

v > u    # 2 < 3   4 == 4   6 > 5
```

TRUE · FALSE · TRUE

FALSE · FALSE · TRUE

Also:

In [15]:
```
u > 4
v*2 > 7
```

FALSE · FALSE · TRUE

FALSE · TRUE · TRUE

Selecting One or More Elements of a Vector:

Depending on the situation, we can use one or more of the following methods:

- Use square brackets to select elements based on their index. (Note that R indices start at 1.)

In [16]:
```
v <- c(0,10,20,30,7,11,23)
v[1]
```

0

- Use negative indices to exclude specific elements.

In [17]:
```
v[-1]
```

10 · 20 · 30 · 7 · 11 · 23

- Use a vector of indices to select multiple elements.

In [18]:
```
v[2:4]
v[c(2,3,4)]
v[-c(2,3,4)]
```

10 · 20 · 30

10 · 20 · 30

0 · 7 · 11 · 23

- Use a logical vector (as we saw earlier, applying logical operators to a vector produces a vector of logical values).

In [19]:
```
v[v > 8]
v[v > median(v)]
```

10 · 20 · 30 · 11 · 23

20 · 30 · 23

Generating a Sequence of Numbers:

- The simplest way to generate a sequence is with an increment of 1:

In [20]:
```
4:9
-3:7
```

4 · 5 · 6 · 7 · 8 · 9

-3 · -2 · -1 · 0 · 1 · 2 · 3 · 4 · 5 · 6 · 7

What do you think the output of `6:4` will be?

In [21]:
```
6:4
```

6 · 5 · 4

- You can use the `seq()` function to create sequences with any increment or to specify the length of the sequence.

In [22]:
```
seq(from = 3, to = 8, by = 2)
seq(from = 5, to = 20, length.out = 4)
```

```
seq(1,2,length.out = 5)
```

3 · 5 · 7

5 · 10 · 15 · 20

1 · 1.25 · 1.5 · 1.75 · 2

- To create repeated values, we use the `rep()` function.

In [23]:
```
rep(2,times = 4)
```

2 · 2 · 2 · 2

In [24]:
```
c(1:5,seq(6,10),(1:5)+10)
```

1 · 2 · 3 · 4 · 5 · 6 · 7 · 8 · 9 · 10 · 11 · 12 · 13 · 14 · 15

Data Structures in R

Vectors:

We discussed vectors earlier. In this section, we cover a few additional tips:

- Adding data to a vector:

In [25]:
```
vec <- 1:6
vec

newVec <- c(vec,10,vec)
newVec

## Inserting Data into a Vector
append(vec,8:10, after = 2)
```

1 · 2 · 3 · 4 · 5 · 6

1 · 2 · 3 · 4 · 5 · 6 · 10 · 1 · 2 · 3 · 4 · 5 · 6

1 · 2 · 8 · 9 · 10 · 3 · 4 · 5 · 6

Recycling Rule in R:

Earlier, we mentioned that when performing operations on vectors in R, the operation is applied element-wise to each pair of elements.

But what happens if the two vectors have different lengths?

In such cases, R applies the **recycling rule** to the shorter vector.
That is, it starts again from the first element of the shorter vector and uses its elements repeatedly as needed.

In [26]:
```r
1:6 + 1:3

## 1 2 3 4 5 6
##   +
## 1 2 3 1 2 3

c(1,2,3,10,20,30) + 1:5

## 1 2 3 10 20 30
##    +
## 1 2 3  4  5  1
```

2 · 4 · 6 · 5 · 7 · 9

```
Warning message in c(1, 2, 3, 10, 20, 30) + 1:5:
“longer object length is not a multiple of shorter object length”
```

2 · 4 · 6 · 14 · 25 · 31

Matrices

One of the data structures available in R is matrices.

In [27]:
```r
## Create a matrix using matrix() function

# 2*4 matrix
matrix(1:8, nrow = 2)

#2*4 matrix, each row is filled first
matrix(1:8,nrow = 2, byrow = T)

# 4*2 matrix
matrix(1:8, ncol = 2)
```

A matrix: 2 × 4 of type int

1	3	5	7
2	4	6	8

A matrix: 2 × 4 of type int

1	2	3	4
5	6	7	8

A matrix: 4 × 2 of type int

1	5
2	6
3	7
4	8

In [28]:
```r
## Name columns and rows of a matrix
m <- matrix(1:4,nrow = 2)
colnames(m) <- c("col one","col two")
rownames(m) <- c("row one","row two")
m
```

A matrix: 2 × 2 of type int

	col one	col two
row one	1	3
row two	2	4

In [29]:
```r
## Calling elements
print(m[2,])
print(m[,2])
print(m[2,2])
```

```
col one col two
      2       4
row one row two
      3       4
[1] 4
```

In [30]:
```r
# Arithmatic on matrices
print("Scalar Multiplication:")
m * 2
print("Element Wise Multiplication:")
m * m
print("Matrix Multiplication:")
m %*% m
```

```
[1] "Scalar Multiplication:"
```

A matrix: 2 × 2 of type dbl

	col one	col two
row one	2	6
row two	4	8

```
[1] "Element Wise Multiplication:"
```

A matrix: 2 × 2 of type int

	col one	col two
row one	1	9
row two	4	16

```
[1] "Matrix Multiplication:"
```

A matrix: 2 × 2 of type dbl

	col one	col two
row one	7	15
row two	10	22

Dataframes:

The dataframe is the most important and widely used data structure in R.

It is a table where each column contains the values of a variable, and each row contains a set of values for each column.

In [31]:
```r
## Create a dataframe with data.frame()
df <- data.frame(
    ID = 1:5,
    Name <- c("Alex", "Ryan", "Kevin", "Nick", "Henry"),
    Grade = c(20,16,17.5,14,19))

df
```

A data.frame: 5 × 3

ID	Name....c..Alex....Ryan....Kevin....Nick....Henry..	Grade
<int>	<chr>	<dbl>
1	Alex	20.0
2	Ryan	16.0
3	Kevin	17.5
4	Nick	14.0
5	Henry	19.0

In [32]:
```
## Summary of a dataframe
summary(df)
```

```
      ID    Name....c..Alex....Ryan....Kevin....Nick....Henry..     Grade      
 Min.   :1   Length:5                                           Min.   :14.0  
 1st Qu.:2   Class :character                                   1st Qu.:16.0  
 Median :3   Mode  :character                                   Median :17.5  
 Mean   :3                                                     Mean   :17.3  
 3rd Qu.:4                                                     3rd Qu.:19.0  
 Max.   :5                                                     Max.   :20.0  
```

In [33]:
```
##Selecting elements
df$Grade
```

20 · 16 · 17.5 · 14 · 19

In [34]:
```
df[1,]
```

A data.frame: 1 × 3

	ID	Name....c..Alex....Ryan....Kevin....Nick....Henry..	Grade
	<int>	<chr>	<dbl>
1	1	Alex	20

In [35]:
```
df[,1]
```

1 · 2 · 3 · 4 · 5

In [36]:
```
df[df$Grade > 18,]
```

A data.frame: 2 × 3

	ID	Name....c..Alex....Ryan....Kevin....Nick....Henry..	Grade
	<int>	<chr>	<dbl>
1	1	Alex	20
5	5	Henry	19

In [37]:
```
df[c("ID","Grade")]
```

A data.frame: 5 × 2

ID	Grade
<int>	<dbl>
1	20.0
2	16.0
3	17.5
4	14.0
5	19.0

Also, when you read a dataset from a file, it is stored as a dataframe.

In [ ]:
```
## Depending on the format of dataset
## for example inorder to read a .csv file, use read.csv()
read.csv("Address of the file")

## Or for a excel file, use read_excel
library(readxl)
read_excel("Address of the file")
```

```r
##And for a sav file, use read_sav
install.packages("haven")
read_sav("Address of the file")
```

Additional Notes

- One of the best ways to quickly access and review the basic concepts of R is by using R cheat sheets.
- Alongside this notebook, a cheat sheet covering basic concepts is provided for your use.
- Additionally, another cheat sheet is included for working with RStudio.

In [ ]: